



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE PROGRAMACIÓN DE COMPUTADORAS

Desarrollo y Gestión de Software

Desarrollo de Software VIII

Laboratorio 2

Manejo e intercambio de datos mediante archivos JSON y CSV en Python

Profesor

José Chiru

Estudiante

Hou, Erick 8-1017-473

Mas, Guillermo 8-1028-2174

Zheng, Jessica 8-1033-370

Grupo

1GS131

Fecha

18/10/2026

Índice

Problema 1 3
Desarrollo 4
Ejecución 7
Problema 2 8
Desarrollo 8
Ejecución 12
Problema 3. Análisis de sensores IoT con Polars 16
Desarrollo 17
Ejecución 19
Conclusión 22

Problema 1

Una institución almacena las calificaciones en estudiantes.csv y necesita enviar los resultados a otro sistema en formato JSON.

cedula, nombre, parcial1, parcial2, proyecto

8-123-456, Marcos,85,90,95

8-234-567, Ana,70,65,80

- Leer al menos 20 registros desde estudiantes.csv.
- Validar notas entre 0 y 100 y evitar cédulas duplicadas.
- Calcular nota final: parcial 1 (30%), parcial 2 (30%) y proyecto (40%).
- Clasificar como APROBADO (nota final >= 71) o REPROBADO.
- Determinar promedio general, mayor y menor nota y cantidades por condición.
- Generar estudiantes.json con los registros, nota final y condición.
- Generar resumen.txt con los indicadores del grupo.

Desarrollo

problema1.py | estudiantes.csv | estudiantes.py | controlEstudiantes.py

problema1 > problema1.py > ...

```python
from utilidades.controlEstudiantes import ControlEstudiante
from utilidades.Reportes import Reportes

archivoEntrada = 'problema1/estudiantes.csv'
archivoSalidaJson = 'problema1/estudiantes.json'
archivoSalidaTxt = 'problema1/resumen.txt'
control = ControlEstudiante(archivoEntrada)

try:
    control.leerArchivo()

    for estudiante in control.estudiantes:
        control.calcularNotaFinal(estudiante)
        control.clasificarEstudiante(estudiante)

    reporte = Reportes(
        control, archivoSalidaJson, archivoSalidaTxt
    )

    reporte.generarEstudiantes()
    reporte.generarResumen()

    print("Archivos de estudiantes generados exitosamente.")

except FileNotFoundError:
    print(f"Error: no se encontró el archivo {archivoEntrada}.")

except Exception as e:
    print(f"Error en la ejecución => {e}")
```

problema1.py | estudiantes.csv | estudiantes.py | controlEstudiantes.py

problema1 > modelos > estudiantes.py > Estudiante > __init__

```python
class Estudiante:
    def __init__(self, cedula, nombre, parcial1, parcial2, proyecto):
        self.nombre = nombre
        self.cedula = cedula
        self.parcial1 = parcial1
        self.parcial2 = parcial2
        self.proyecto = proyecto
        self.notaFinal = 0.0
        self.clasificacion = ""

```

problema1.py | estudiantes.csv | estudiantes.py | controlEstudiantes.py | Reportes.py

problema1 > utilidades > controlEstudiantes.py > ControlEstudiante > determinarMayorNota

```python
import csv
from modelos.estudiantes import Estudiante

class ControlEstudiante:
    def __init__(self, archivoEntrada):
        self.archivoEntrada = archivoEntrada
        self.estudiantes = []
        self.cedulas = []

    def leerArchivo(self):
        with open(self.archivoEntrada, 'r', encoding='utf-8', newline='') as archivo:
            lector = csv.reader(archivo)
            next(lector, None) #Omite el encabezado del archivo CSV

            for datos in lector:
                if len(datos) == 5:
                    cedula, nombre, parcial1, parcial2, proyecto = datos
                    #Valida que las notas sean números enteros
                    try:
                        parcial1 = int(parcial1)
                        parcial2 = int(parcial2)
                        proyecto = int(proyecto)
                    except ValueError:
                        raise ValueError("Las notas deben ser números enteros.")

                    notas = [parcial1, parcial2, proyecto]
                    self.validarNota(notas)
                    self.validarCedula(cedula)
                    self.cedulas.append(cedula)
                    self.estudiantes.append(Estudiante(cedula, nombre, parcial1, parcial2, proyecto))
                else:
                    raise ValueError("Un registro no contiene cinco campos.")
```

problema1.py | estudiantes.csv | estudiantes.py | controlEstudiantes.py | Reportes.py

problema1 > utilidades > Reportes.py > Reportes > generarEstudiantes

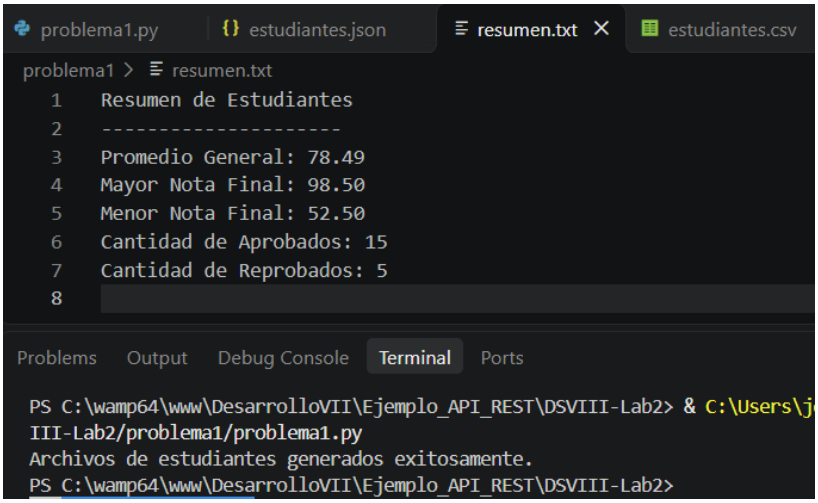
```python
import json

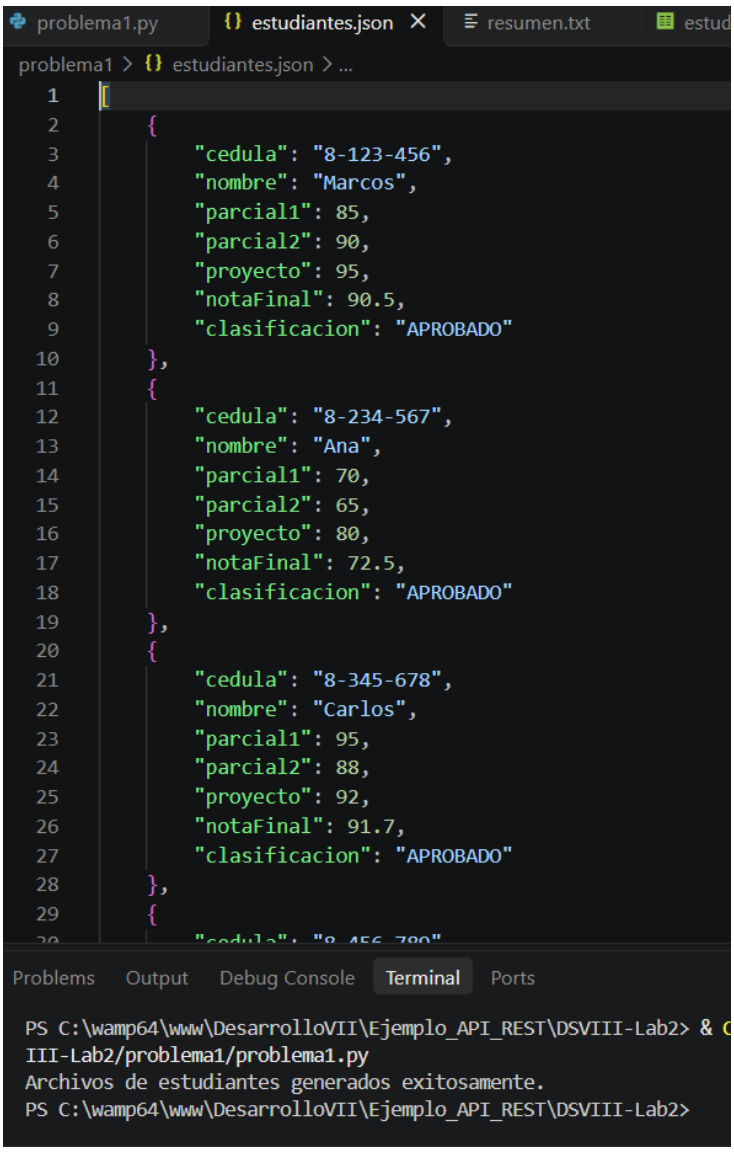
class Reportes:
    def __init__(self, control, archivoSalidaJson, archivoSalidaTxt):
        self.control = control
        self.archivoSalidaJson = archivoSalidaJson
        self.archivoSalidaTxt = archivoSalidaTxt

    #Genera un archivo JSON con los registros, nota final y condición de los estudiantes.
    def generarEstudiantes(self):
        datos = []
        for estudiante in self.control.estudiantes:
            registro = {
                "cedula": estudiante.cedula,
                "nombre": estudiante.nombre,
                "parcial1": estudiante.parcial1,
                "parcial2": estudiante.parcial2,
                "proyecto": estudiante.proyecto,
                "notaFinal": estudiante.notaFinal,
                "clasificacion": estudiante.clasificacion
            }
            datos.append(registro)
        with open(self.archivoSalidaJson, 'w', encoding='utf-8') as archivo:
            json.dump(datos, archivo, indent=4, ensure_ascii=False)

    #Genera un archivo txt con los indicadores del grupo.
    def generarResumen(self):
        promedio = self.control.calcularPromedioGeneral()
        mayor = self.control.determinarMayorNota()
        menor = self.control.determinarMenorNota()
        aprobados, reprobados = self.control.contarClasificados()
        with open(self.archivoSalidaTxt, 'w', encoding='utf-8') as archivo:
            archivo.write(f"Resumen de Estudiantes\n")
            archivo.write(f"----------------------\n")
            archivo.write(f"Promedio General: {promedio:.2f}\n")
            archivo.write(f"Mayor Nota Final: {mayor:.2f}\n")
```

Ejecución


problema1.py
estudiantes.json
resumen.txt
estudiantes.csv
problema1 > resumen.txt
1 Resumen de Estudiantes
2 ----------
3 Promedio General: 78.49
4 Mayor Nota Final: 98.50
5 Menor Nota Final: 52.50
6 Cantidad de Aprobados: 15
7 Cantidad de Reprobados: 5
8
Problems
Output
Debug Console
Terminal
Ports
PS C:\wamp64\www\DesarrolloVII\Ejemplo_API_REST\DSVIII-Lab2> & C:\Users\j
III-Lab2/problema1/problema1.py
Archivos de estudiantes generados exitosamente.
PS C:\wamp64\www\DesarrolloVII\Ejemplo_API_REST\DSVIII-Lab2>


problema1.py
estudiantes.json
resumen.txt
problema1 > estudiantes.json > ...
[
{
"cedula": "8-123-456",
"nombre": "Marcos",
"parcial1": 85,
"parcial2": 90,
"proyecto": 95,
"notaFinal": 90.5,
"clasificacion": "APROBADO"
},
{
"cedula": "8-234-567",
"nombre": "Ana",
"parcial1": 70,
"parcial2": 65,
"proyecto": 80,
"notaFinal": 72.5,
"clasificacion": "APROBADO"
},
{
"cedula": "8-345-678",
"nombre": "Carlos",
"parcial1": 95,
"parcial2": 88,
"proyecto": 92,
"notaFinal": 91.7,
"clasificacion": "APROBADO"
},
{
Problems
Output
Debug Console
Terminal
Ports
PS C:\wamp64\www\DesarrolloVII\Ejemplo_API_REST\DSVIII-Lab2> &
III-Lab2/problema1/problema1.py
Archivos de estudiantes generados exitosamente.
PS C:\wamp64\www\DesarrolloVII\Ejemplo_API_REST\DSVIII-Lab2>

Problema 2

Una empresa recibe su inventario en JSON y debe actualizarlo y enviarlo a otra sucursal en CSV.

[

{"codigo":"P001","producto":"Teclado","cantidad":25,"precio":18.50},

{"codigo":"P002","producto":"Mouse","cantidad":40,"precio":12.75}

]

- Cargar al menos 20 productos desde inventario.json.
- Presentar un menú para listar, buscar por código y registrar entradas o salidas.
- Impedir existencias negativas, cantidades de movimiento no positivas y códigos duplicados.
- Calcular el valor total del inventario e identificar productos con menos de 15 unidades.
- Guardar inventario_actualizado.json e inventario.csv con los cambios.
- Generar movimientos.csv con código, tipo, cantidad y existencia final.

Desarrollo

problema2.py | inventario.json | movimientos.csv | inventario.csv

problema2 > problema2.py > registrar_salida

```python
import csv
import json
import os

# funciones de respaldo

def LeerJson(archivo):
    with open(archivo, "r") as archivo_json:
        inventario = json.load(archivo_json)
    return inventario

def guardarJson(ruta, datos):
    with open(ruta, "w", encoding="utf-8") as f:
        json.dump(datos, f, indent=4, ensure_ascii=False)

def iniciar_inventario():

    if not os.path.exists(ARCHIVO_INVENTARIO):
        raise FileNotFoundError(f"No existe {ARCHIVO_INVENTARIO}")

    if not os.path.exists(ARCHIVO_ACTUALIZADO) or os.path.getsize(ARCHIVO_ACTUALIZADO) == 0:
        datos_originales = LeerJson(ARCHIVO_INVENTARIO)
        guardarJson(ARCHIVO_ACTUALIZADO, datos_originales)

    return LeerJson(ARCHIVO_ACTUALIZADO)

def capturar_movimiento(movimientos, codigo, cantidad, tipo, existencia_final=None):
    """Agrega un movimiento ('entrada' o 'salida') a la lista compartida."""
    if tipo not in ("entrada", "salida"):
        raise ValueError("El tipo debe ser 'entrada' o 'salida'")
    movimientos.append({"codigo": codigo, "cantidad": cantidad, "tipo": tipo, "existencia final": existencia_final})


def exportar_movimientos(movimientos, archivo_csv):

    with open(archivo_csv, "w", newline="", encoding="utf-8") as csvfile:
        writer = csv.writer(csvfile)
        writer.writerow(["codigo", "tipo", "cantidad", "existencia final"])
        for m in movimientos:
```

problema2.py | inventario.json | movimientos.csv | inventario.csv

problema2 > problema2.py > registrar_salida

```python
def exportar_invAc_como_csv(inventario, archivo_csv):
    with open(archivo_csv, "w", newline="", encoding="utf-8") as csvfile:
        writer = csv.writer(csvfile)
        writer.writerow(["codigo", "producto", "cantidad", "precio"])
        for producto in inventario:
            writer.writerow([producto["codigo"], producto["producto"], producto["cantidad"], producto["precio"]])

# utilidad

def listar_productos(inventario):
    print("Listado de productos:")
    for producto in inventario:
        print(f"Código: {producto['codigo']}, producto: {producto['producto']}, cantidad: {producto['cantidad']}, Precio: {producto['precio']}")

def obtenerProductoPorCodigo(inventario, codigo):
    for producto in inventario:
        if producto['codigo'] == codigo:
            return producto
    return None

def registrar_entrada(inventario, codigo, cantidad):
    producto = obtenerProductoPorCodigo(inventario, codigo)
    cantidadActual = producto['cantidad'] if producto else 0
    if producto:
        if cantidad < 0:
            print("Error: La cantidad de entrada no puede ser negativa.")
            return
        producto['cantidad'] += cantidad
        print(f"\nEntrada registrada: {producto['producto']}, Cantidad: {cantidad}, Existencia actual: {cantidadActual + cantidad}")
        capturar_movimiento(movimientos, codigo, cantidad, "entrada", cantidadActual + cantidad)
    else:
        print("Producto no encontrado.")

def registrar_salida(inventario, codigo, cantidad):
    if cantidad < 0:
        print("Error: La cantidad de salida no puede ser negativa.")
        return
    producto = obtenerProductoPorCodigo(inventario, codigo)
    cantidadActual = producto['cantidad'] if producto else 0
```

problema2.py | inventario.json | movimientos.csv | inventario.csv

problema2 > problema2.py > registrar_salida

```python
        print("Producto no encontrado.")

def registrar_salida(inventario, codigo, cantidad):
    if cantidad < 0:
        print("Error: La cantidad de salida no puede ser negativa.")
        return
    producto = obtenerProductoPorCodigo(inventario, codigo)
    cantidadActual = producto['cantidad'] if producto else 0
    if producto:
        if producto['cantidad'] >= cantidad:
            producto['cantidad'] -= cantidad
            print(f"\nSalida registrada: {producto['producto']}, Cantidad: {cantidad}, Existencia actual: {cantidadActual - cantidad}")
            capturar_movimiento(movimientos, codigo, cantidad, "salida", cantidadActual - cantidad)
        else:
            print("Error: No hay suficiente stock para realizar la salida.")
    else:
        print("Producto no encontrado.")

def calcular_total_inventario(inventario):
    total = sum(producto['cantidad'] * producto['precio'] for producto in inventario)
    return total

def calcular_productos_bajos_stock(inventario, umbral=15):
    productos_bajos = [producto for producto in inventario if producto['cantidad'] < umbral]
    return productos_bajos


ARCHIVO_INVENTARIO = "problema2/inventario.json"
ARCHIVO_ACTUALIZADO = "problema2/inventario_actualizado.json"
ARCHIVO_CSV = "problema2/inventario.csv"
ARCHIVO_MOVIMIENTOS = "problema2/movimientos.csv"
movimientos = []

try:
    inventario = iniciar_inventario()
except FileNotFoundError:
    print(f"Error al leer los archivos {ARCHIVO_INVENTARIO} o {ARCHIVO_ACTUALIZADO}. Asegúrese de que existan.")
```

problema2.py | inventario.json | movimientos.csv | inventario.csv

problema2 > problema2.py > registrar_salida

```
105
106 try:
107     inventario = iniciar_inventario()
108 except FileNotFoundError:
109     print(f"Error al leer los archivos {ARCHIVO_INVENTARIO} o {ARCHIVO_ACTUALIZADO}. Asegúrese de que existan.")
110
111
112 print("Bienvenido al sistema de inventario de la tienda\n.")
113 while True:
114     print(
115         "\nSeleccione una de las siguientes opciones:\n"
116         "1. Listar productos\n"
117         "2. Buscar producto por código\n"
118         "3. Registrar entrada de productos\n"
119         "4. Registrar salida de productos\n"
120         "5. Mostrar Cálculos (Total de Inventario, Productos con menos de 15 unidades)\n"
121         "6. Exportar Movimientos\n"
122         "7. Guardar y Salir del sistema de inventario"
123     )
124
125     opcion = input("Ingrese el número de la opción deseada: ")
126
127     match opcion:
128         case "1":
129             listar_productos(inventario)
130         case "2":
131             producto = obtenerProductoPorCodigo(
132                 inventario,
133                 input("Ingrese el código del producto a buscar: ")
134             )
135             if producto:
136                 print(
137                     f"Producto encontrado: {producto['producto']}, "
138                     f"Cantidad: {producto['cantidad']}, Precio: {producto['precio']}"
139                 )
140             else:
141                 print("Producto no encontrado.")
142         case "3":
143             registrar_entrada(
```

Problems | Output | Debug Console | Terminal | Ports

problema2.py | inventario.json | movimientos.csv | inventario.csv

problema2 > problema2.py > registrar_salida

```
                    int(input("Ingrese la cantidad a registrar: "))
                )
            case "4":
                registrar_salida(
                    inventario,
                    input("Ingrese el código del producto: "),
                    int(input("Ingrese la cantidad a registrar: "))
                )
            case "5":
                total_inventario = calcular_total_inventario(inventario)
                productos_bajos_stock = calcular_productos_bajos_stock(inventario)
                print(f"Total de inventario: {total_inventario}")
                print("Productos con menos de 15 unidades:\n")
                for producto in productos_bajos_stock:
                    print(f"- {producto['producto']}: {producto['cantidad']} unidades")

            case "6":
                exportar_movimientos(movimientos, ARCHIVO_MOVIMIENTOS)
                print("exportando movimientos a CSV...")

            case "7":
                print("\nGuardando cambios en inventario.csv...")
                exportar_invAc_como_csv(inventario, ARCHIVO_CSV)
                print(f"Inventario guardado en {ARCHIVO_CSV}.\n")

                print("\nGuardando cambios en inventario.json...")
                guardarJson(ARCHIVO_INVENTARIO, inventario)
                print(f"Inventario guardado en {ARCHIVO_INVENTARIO}.")

                if os.path.exists(ARCHIVO_ACTUALIZADO):
                    os.remove(ARCHIVO_ACTUALIZADO)
                    print(f"Archivo temporal eliminado: {ARCHIVO_ACTUALIZADO}.")

                print("Saliendo del sistema de inventario. ¡Hasta luego!")

                break
            case _:
                print("Opción no válida. Seleccione una opción del 1 al 8.")

```

Ejecución

```
Ingrese el número de la opción deseada: 1
Listado de productos:
Código: P001, producto: Teclado Dell KB216, cantidad: 25, Precio: 18.5
Código: P002, producto: Mouse HP X1000, cantidad: 40, Precio: 12.75
Código: P003, producto: Monitor Samsung 24 pulgadas, cantidad: 15, Precio: 149.99
Código: P004, producto: Laptop Lenovo IdeaPad 3, cantidad: 10, Precio: 579.99
Código: P005, producto: Celular Samsung Galaxy A15, cantidad: 18, Precio: 199.99
Código: P006, producto: Tablet Lenovo Tab M10, cantidad: 12, Precio: 169.5
Código: P007, producto: Monitor LG UltraGear 27 pulgadas, cantidad: 8, Precio: 299.99
Código: P008, producto: Impresora HP DeskJet 2775, cantidad: 7, Precio: 89.99
Código: P009, producto: Audifonos Sony WH-CH520, cantidad: 22, Precio: 59.99
Código: P010, producto: Bocina RCA Bluetooth, cantidad: 20, Precio: 34.95
Código: P011, producto: Televisor LG Smart TV 43 pulgadas, cantidad: 6, Precio: 329.99
Código: P012, producto: Disco duro externo Seagate 1TB, cantidad: 14, Precio: 64.99
Código: P013, producto: Memoria USB Kingston 64GB, cantidad: 35, Precio: 9.99
Código: P014, producto: Webcam Logitech C920, cantidad: 11, Precio: 69.99
Código: P015, producto: Router TP-Link Archer C6, cantidad: 13, Precio: 49.99
Código: P016, producto: Cargador Anker USB-C, cantidad: 28, Precio: 24.99
Código: P017, producto: Smartwatch Xiaomi Redmi Watch 3, cantidad: 16, Precio: 79.99
Código: P018, producto: Consola Nintendo Switch Lite, cantidad: 5, Precio: 199.99
Código: P019, producto: Control Sony PlayStation 5, cantidad: 9, Precio: 69.99
Código: P020, producto: Microfono HyperX SoloCast, cantidad: 10, Precio: 59.99
Código: P021, producto: Camara Canon EOS Rebel T7, cantidad: 4, Precio: 449.99
Código: P022, producto: Proyector Epson EpiqVision, cantidad: 3, Precio: 399.99
Código: P023, producto: Bateria portatil Belkin 10000mAh, cantidad: 19, Precio: 29.99
Código: P024, producto: Adaptador USB-C UGREEN, cantidad: 24, Precio: 19.99
Código: P025, producto: Lector Kindle Paperwhite, cantidad: 7, Precio: 139.99
Código: P026, producto: Reproductor Roku Express, cantidad: 15, Precio: 29.99
Código: P027, producto: Monitor Dell P2422H, cantidad: 9, Precio: 219.99
Código: P028, producto: Laptop HP Pavilion 15, cantidad: 8, Precio: 649.99
Código: P029, producto: Celular Motorola Moto G54, cantidad: 14, Precio: 249.99
Código: P030, producto: Televisor Samsung Crystal UHD 50 pulgadas, cantidad: 57, Precio: 429.99
```

```
Ingrese el número de la opción deseada: 2
Ingrese el código del producto a buscar: P030
Producto encontrado: Televisor Samsung Crystal UHD 50 pulgadas, Cantidad: 57, Precio: 429.99
```

```
Seleccione una de las siguientes opciones:
1. Listar productos
2. Buscar producto por código
3. Registrar entrada de productos
4. Registrar salida de productos
5. Mostrar Cálculos (Total de Inventario, Productos con menos de 15 unidades)
6. Exportar Movimientos
7. Guardar y Salir del sistema de inventario
Ingrese el número de la opción deseada: 3
Ingrese el código del producto: P030
Ingrese la cantidad a registrar: -1
Error: La cantidad de entrada no puede ser negativa.
```

```
Seleccione una de las siguientes opciones:
1. Listar productos
2. Buscar producto por código
3. Registrar entrada de productos
4. Registrar salida de productos
5. Mostrar Cálculos (Total de Inventario, Productos con menos de 15 unidades)
6. Exportar Movimientos
7. Guardar y Salir del sistema de inventario
Ingrese el número de la opción deseada: 3
Ingrese el código del producto: P030
Ingrese la cantidad a registrar: 19

Entrada registrada: Televisor Samsung Crystal UHD 50 pulgadas, Cantidad: 19, Existencia actual: 76
```

```
Seleccione una de las siguientes opciones:
1. Listar productos
2. Buscar producto por código
3. Registrar entrada de productos
4. Registrar salida de productos
5. Mostrar Cálculos (Total de Inventario, Productos con menos de 15 unidades)
6. Exportar Movimientos
7. Guardar y Salir del sistema de inventario
Ingrese el número de la opción deseada: 4
Ingrese el código del producto: P003
Ingrese la cantidad a registrar: 5

Salida registrada: Monitor Samsung 24 pulgadas, Cantidad: 5, Existencia actual: 10
```

```
Ingrese el número de la opción deseada: 5
Total de inventario: 76661.49
Productos con menos de 15 unidades:

- Monitor Samsung 24 pulgadas: 10 unidades
- Laptop Lenovo IdeaPad 3: 10 unidades
- Tablet Lenovo Tab M10: 12 unidades
- Monitor LG UltraGear 27 pulgadas: 8 unidades
- Impresora HP DeskJet 2775: 7 unidades
- Televisor LG Smart TV 43 pulgadas: 6 unidades
- Disco duro externo Seagate 1TB: 14 unidades
- Webcam Logitech C920: 11 unidades
- Router TP-Link Archer C6: 13 unidades
- Consola Nintendo Switch Lite: 5 unidades
- Control Sony PlayStation 5: 9 unidades
- Microfono HyperX SoloCast: 10 unidades
- Camara Canon EOS Rebel T7: 4 unidades
- Proyector Epson EpiqVision: 3 unidades
- Lector Kindle Paperwhite: 7 unidades
- Monitor Dell P2422H: 9 unidades
- Laptop HP Pavilion 15: 8 unidades
- Celular Motorola Moto G54: 14 unidades
```

```
Ingrese el número de la opción deseada: 6
exportando movimientos a CSV...
```

problema2.py ● {} inventario.json movimientos.csv M ✕ inventario.csv

problema2 > movimientos.csv

```
1  codigo,tipo,cantidad,existencia final
2  P030,entrada,19,76
3  P003,salida,5,10
4
```

```
7. Guardar y Salir del sistema de inventario
Ingrese el número de la opción deseada: 7

Guardando cambios en inventario.csv...
Inventario guardado en problema2/inventario.csv.


Guardando cambios en inventario.json...
Inventario guardado en problema2/inventario.json.
Archivo temporal eliminado: problema2/inventario_actualizado.json.
Saliendo del sistema de inventario. ¡Hasta luego!
PS D:\Coding\DS8\Lab2\DSVIII-Lab2>
```

problema2 > inventario.csv

```
codigo,producto,cantidad,precio
P001,Teclado Dell KB216,25,18.5
P002,Mouse HP X1000,40,12.75
P003,Monitor Samsung 24 pulgadas,10,149.99
P004,Laptop Lenovo IdeaPad 3,10,579.99
P005,Celular Samsung Galaxy A15,18,199.99
P006,Tablet Lenovo Tab M10,12,169.5
P007,Monitor LG UltraGear 27 pulgadas,8,299.99
P008,Impresora HP DeskJet 2775,7,89.99
P009,Audifonos Sony WH-CH520,22,59.99
P010,Bocina RCA Bluetooth,20,34.95
P011,Televisor LG Smart TV 43 pulgadas,6,329.99
P012,Disco duro externo Seagate 1TB,14,64.99
P013,Memoria USB Kingston 64GB,35,9.99
P014,Webcam Logitech C920,11,69.99
P015,Router TP-Link Archer C6,13,49.99
P016,Cargador Anker USB-C,28,24.99
P017,Smartwatch Xiaomi Redmi Watch 3,16,79.99
P018,Consola Nintendo Switch Lite,5,199.99
P019,Control Sony PlayStation 5,9,69.99
P020,Microfono HyperX SoloCast,10,59.99
P021,Camara Canon EOS Rebel T7,4,449.99
P022,Proyector Epson EpiqVision,3,399.99
P023,Bateria portatil Belkin 10000mAh,19,29.99
P024,Adaptador USB-C UGREEN,24,19.99
P025,Lector Kindle Paperwhite,7,139.99
P026,Reproductor Roku Express,15,29.99
P027,Monitor Dell P2422H,9,219.99
P028,Laptop HP Pavilion 15,8,649.99
P029,Celular Motorola Moto G54,14,249.99
P030,Televisor Samsung Crystal UHD 50 pulgadas,76,429.99
```

Problema 3. Análisis de sensores IoT con Polars

Una plataforma recibe mediciones ambientales desde dispositivos IoT y requiere generar estadísticas y alertas intercambiables con otros sistemas.

sensor,ubicacion,fecha,hora,temperatura,humedad

S01,Edificio A,2026-10-01,08:00,28.5,75

S02,Edificio B,2026-10-01,08:00,30.2,80

- Leer con Polars un archivo sensores.csv de al menos 20 mediciones de varios sensores.
- Validar identificadores, temperaturas, humedad entre 0 y 100 y registros incompletos.
- Calcular cantidad de mediciones y temperatura promedio, máxima y mínima por sensor.
- Calcular humedad promedio por sensor y ubicación.
- Crear la columna estado: ALERTA si temperatura > 32.0 °C o humedad > 80%; NORMAL en otro caso.
- Filtrar las alertas e incluir el motivo que activó cada una.
- Exportar alertas.json, resumen_sensores.csv y sensores_procesados.json.
- Utilizar las operaciones de filtrado, agrupación, agregación y creación de columnas de Polars para procesar los datos.

Desarrollo

problema3.py | medicion.py | controlMedicion.py | sensores.csv | resumen_sensores.csv M

problema3 > modelos > medicion.py > Medicion

```python
class Medicion:
    def __init__(self, sensor, ubicacion, fecha, hora, temperatura, humedad, linea=None):
        self.sensor = sensor
        self.ubicacion = ubicacion
        self.fecha = fecha
        self.hora = hora
        self.temperatura = temperatura
        self.humedad = humedad
        self.linea = linea  # número de línea en el CSV, para reportar errores

    def to_dict(self):
        return {
            "linea": self.linea,
            "sensor": self.sensor,
            "ubicacion": self.ubicacion,
            "fecha": self.fecha,
            "hora": self.hora,
            "temperatura": self.temperatura,
            "humedad": self.humedad,
        }

    def __repr__(self):
        return (
            f"Medicion({self.sensor}, {self.ubicacion}, {self.fecha} {self.hora}, "
            f"temp={self.temperatura}, hum={self.humedad})"
        )
```

problema3.py | controlMedicion.py | sensores.csv | resumen_sensores.csv M | sensores_procesados.json

problema3 > utilidades > controlMedicion.py > ControlMedicion > leer_mediciones

```python
import json
from pathlib import Path

import polars as pl

from modelos.medicion import Medicion


class ControlMedicion:
    COLUMNAS_REQUERIDAS = ("sensor", "ubicacion", "fecha", "hora", "temperatura", "humedad")
    PATRON_SENSOR = r"^S\d{2}$"  # S01, S02, ... S99
    HUM_MIN, HUM_MAX = 0.0, 100.0
    TEMP_ALERTA = 32.0  # ALERTA si temperatura > 32.0 °C
    HUM_ALERTA = 80.0   # ALERTA si humedad > 80 %

    def __init__(self, archivoEntrada, archivoAlertas, archivoResumen, archivoSProcesados):
        self.archivoEntrada = archivoEntrada
        self.archivoAlertas = archivoAlertas
        self.archivoResumen = archivoResumen
        self.archivoSProcesados = archivoSProcesados
        self.mediciones = []    # lista de objetos Medicion
        self.validas = None     # DataFrame con los registros que pasaron la validación
        self.invalidos = None   # DataFrame con los rechazados y su motivo
        self.procesados = None  # validas + columna estado
        self.alertas = None     # solo ALERTA, con columna motivo
        self.resumen = None     # estadísticas por sensor

    # 1. Lectura: CSV -> lista de Medicion (sin validar)
```

problema3.py | sensores.csv | resumen_sensores.csv M | sensores_procesados.json | alertas.json

problema3 > problema3.py > ...

```python
from utilidades.controlMedicion import ControlMedicion

control = ControlMedicion(
    archivoEntrada="sensores.csv",
    archivoAlertas="alertas.json",
    archivoResumen="resumen_sensores.csv",
    archivoSProcesados="sensores_procesados.json",
)

    # 1. Leer el CSV y guardar las mediciones en una lista de objetos Medicion (antes de ejecutar, hacer
control.leer_mediciones()

    # 2. Validar con Polars (identificadores, humedad 0-100, registros incompletos)
control.validar_mediciones()

    # 3. Calcular estado, alertas y resumen
control.procesar()

    # 4. Exportar alertas.json, resumen_sensores.csv y sensores_procesados.json
control.exportar()

    # Reporte en consola
print(
    f"Leídas: {len(control.mediciones)} | válidas: {control.validas.height} "
    f"| rechazadas: {control.invalidos.height}"
)
if control.invalidos.height:
    print("\nRegistros rechazados:")
```

sensores.csv | resumen_sensores.csv M | sensores_procesados.json | alertas.json

problema3 > sensores.csv

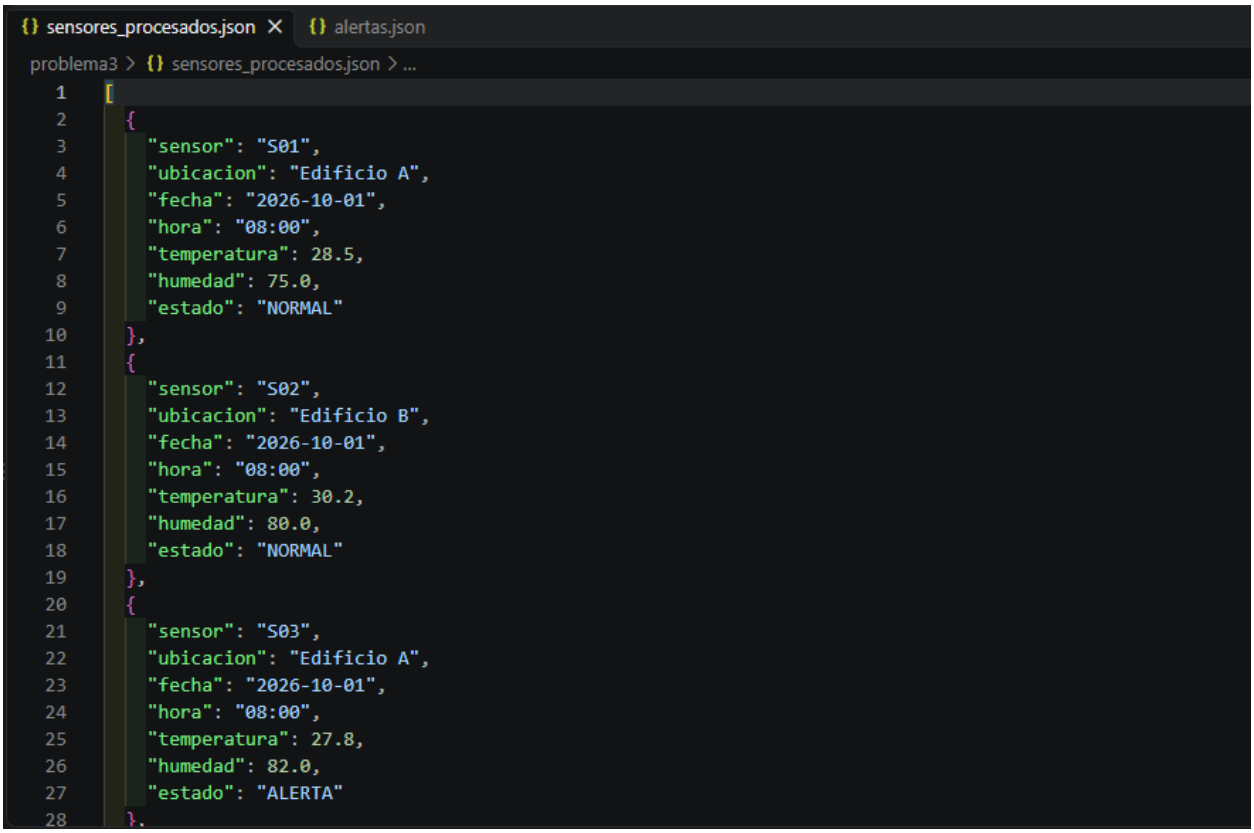
```
sensor,ubicacion,fecha,hora,temperatura,humedad
S01,Edificio A,2026-10-01,08:00,28.5,75
S02,Edificio B,2026-10-01,08:00,30.2,80
S03,Edificio A,2026-10-01,08:00,27.8,82
S05,Edificio E,2026-10-01,08:00,29.1,78
S04,Edificio B,2026-10-01,08:00,26.9,69
S02,Edificio B,2026-10-02,12:00,32.4,62
S01,Edificio A,2026-10-02,12:00,33.1,85
S02,Edificio B,2026-10-02,12:00,31.7,60
S04,Edificio B,2026-10-02,12:00,34.0,58
S03,Edificio A,2026-10-02,12:00,30.5,94
S05,Edificio E,2026-10-02,18:00,29.3,74
S01,Edificio A,2026-10-03,18:00,30.8,77
S03,Edificio A,2026-10-03,18:00,28.6,79
S02,Edificio B,2026-10-03,08:00,28.2,82
S05,Edificio E,2026-10-03,08:00,27.4,85
S03,Edificio A,2026-10-03,08:00,27.9,80
S02,Edificio B,2026-10-03,08:00,29.5,83
S04,Edificio B,2026-10-03,12:00,33.6,59
S01,Edificio A,2026-10-04,12:00,35.2,55
S05,Edificio E,2026-10-04,12:00,31.9,61
S01,Edificio A,2026-10-04,12:00,32.8,93
S02,Edificio B,2026-10-04,18:00,31.2,70
S04,Edificio B,2026-10-04,18:00,29.7,73
S02,Edificio B,2026-10-04,18:00,30.4,71
S05,Edificio E,2026-10-04,18:00,28.8,76
S01,Edificio A,2026-10-04,18:00,29.9,75
```

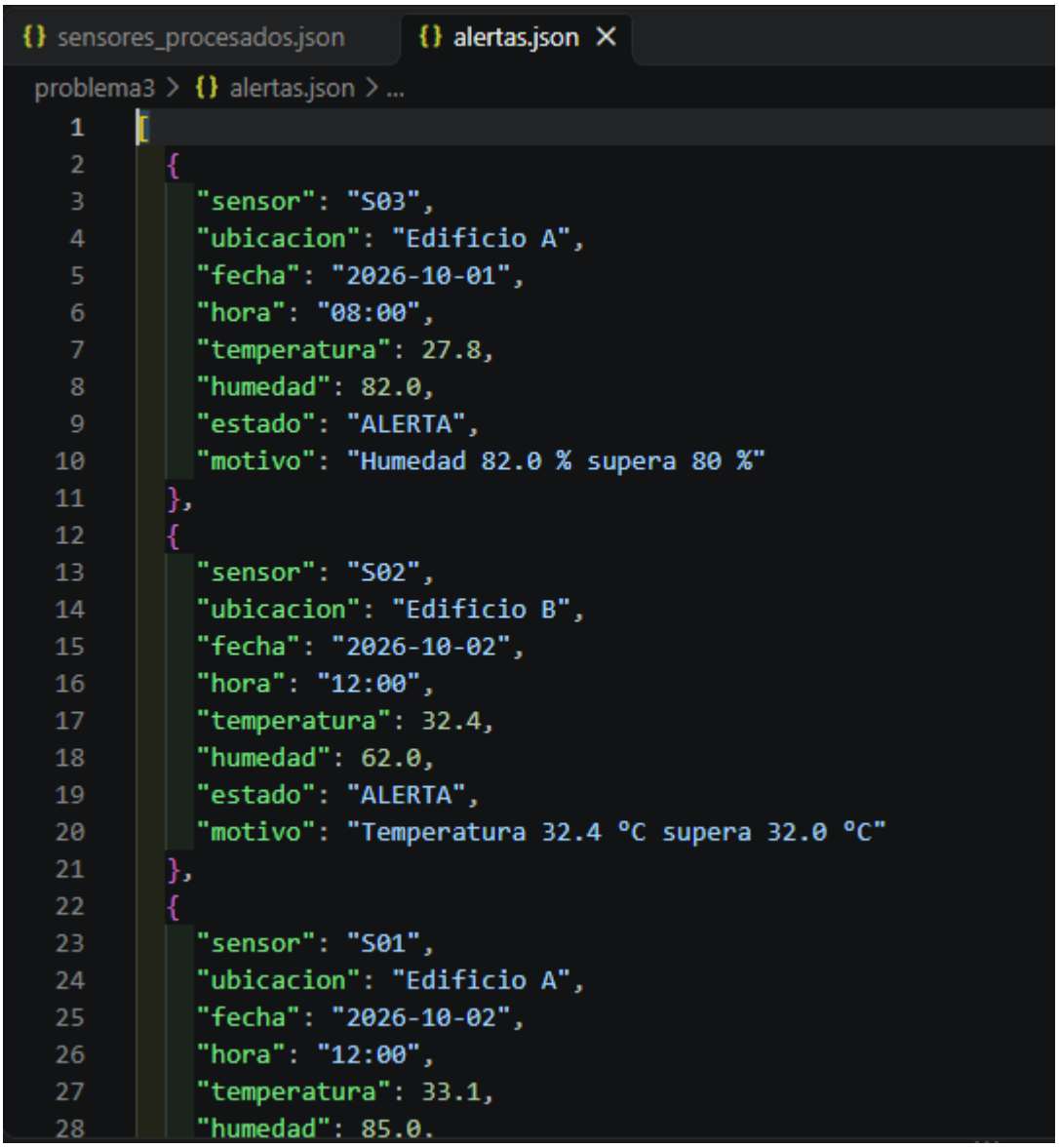
Ejecución

resumen_sensores.csv M | sensores_procesados.json | alertas.json

problema3 > resumen_sensores.csv

```
sensor,ubicacion,cantidad_mediciones,temperatura_promedio,temperatura_maxima,temperatura_minima,humedad_p
S01,Edificio A,6,31.72,35.2,28.5,76.67
S02,Edificio B,7,30.51,32.4,28.2,72.57
S03,Edificio A,5,30.7,38.7,27.8,83.8
S04,Edificio B,4,31.05,34.0,26.9,64.75
S05,Edificio E,5,29.3,31.9,27.4,74.8
```


sensores_procesados.json
alertas.json
problema3 > sensores_procesados.json > ...
[
{
"sensor": "S01",
"ubicacion": "Edificio A",
"fecha": "2026-10-01",
"hora": "08:00",
"temperatura": 28.5,
"humedad": 75.0,
"estado": "NORMAL"
},
{
"sensor": "S02",
"ubicacion": "Edificio B",
"fecha": "2026-10-01",
"hora": "08:00",
"temperatura": 30.2,
"humedad": 80.0,
"estado": "NORMAL"
},
{
"sensor": "S03",
"ubicacion": "Edificio A",
"fecha": "2026-10-01",
"hora": "08:00",
"temperatura": 27.8,
"humedad": 82.0,
"estado": "ALERTA"
},


{} sensores_procesados.json
{} alertas.json
problema3 > {} alertas.json > ...
[
{
"sensor": "S03",
"ubicacion": "Edificio A",
"fecha": "2026-10-01",
"hora": "08:00",
"temperatura": 27.8,
"humedad": 82.0,
"estado": "ALERTA",
"motivo": "Humedad 82.0 % supera 80 %"
},
{
"sensor": "S02",
"ubicacion": "Edificio B",
"fecha": "2026-10-02",
"hora": "12:00",
"temperatura": 32.4,
"humedad": 62.0,
"estado": "ALERTA",
"motivo": "Temperatura 32.4 °C supera 32.0 °C"
},
{
"sensor": "S01",
"ubicacion": "Edificio A",
"fecha": "2026-10-02",
"hora": "12:00",
"temperatura": 33.1,
"humedad": 85.0.

Conclusión

La conversión entre formatos (CSV a JSON y viceversa) mostró que se puede cambiar la forma de los datos sin perder su integridad, siempre que se respeten los tipos (números, textos, fechas) y la estructura de cada registro. Los archivos generados (estudiantes.json, inventario.csv, movimientos.csv, alertas.json, resumen_sensores.csv, entre otros) quedan listos para que otro sistema los consuma sin pasos manuales.

Cabe recalcar que en el análisis de sensores IoT con Polars se mostró otra forma de procesar datos: en lugar de recorrer registro por registro, se usan operaciones de filtrado, agrupación, agregación y creación de columnas. Así se obtuvieron los promedios, máximos y mínimos por sensor, el estado de cada medición (ALERTA o NORMAL) y el motivo de cada alerta con un código más corto y legible, y adecuado para volúmenes de datos mayores.

En conjunto, estos ejercicios reforzaron una forma de trabajo ordenada, que consiste en separar la lectura, la validación, el procesamiento y la exportación, y mostraron que Python y Polars sirven para automatizar el intercambio de información entre sistemas con datos confiables.